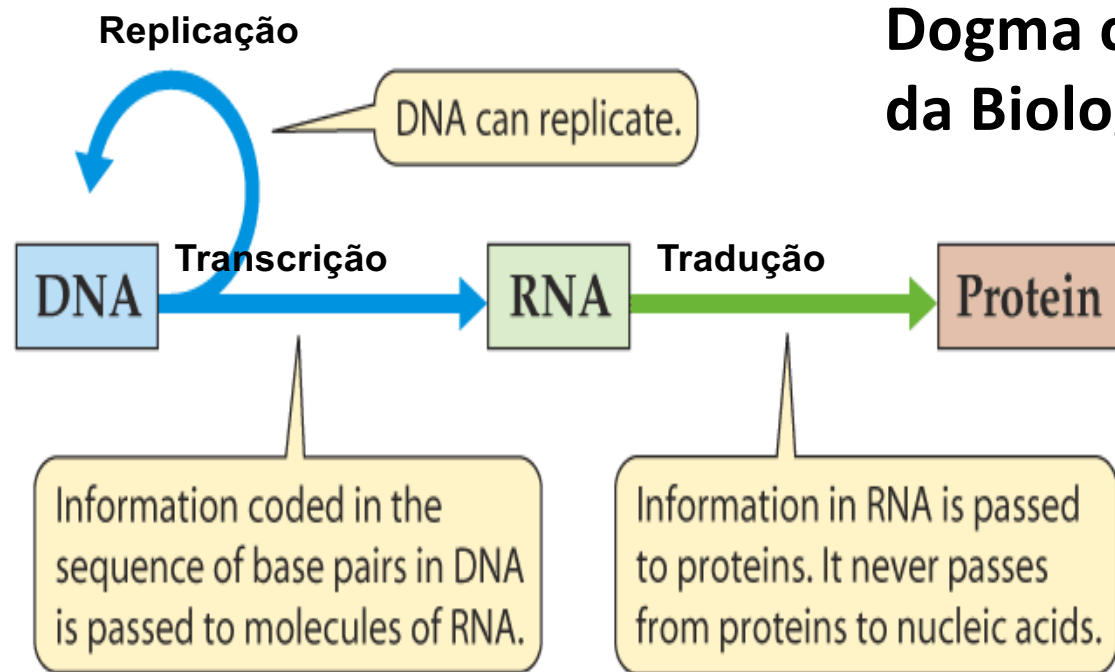


Bioinformática

Revisões de programação em Python
Parte 2

DNA, RNA e o fluxo de informação

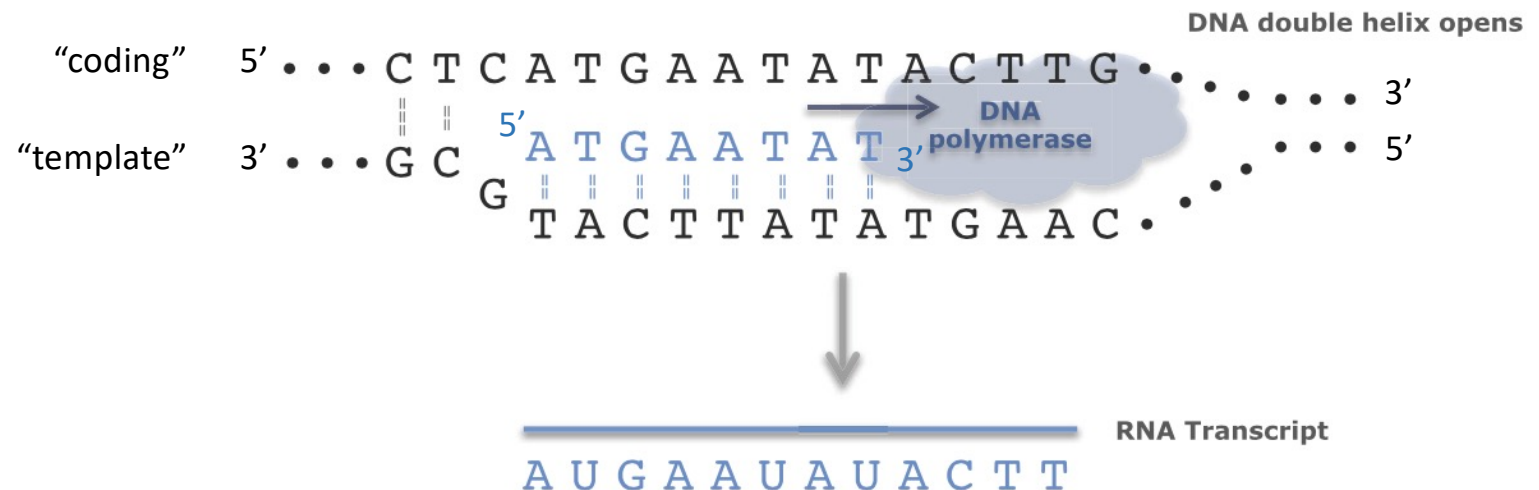


**Dogma central
da Biologia**

Transcrição

A **transcrição** consta da conversão de uma sequência de DNA, numa sequência de RNA

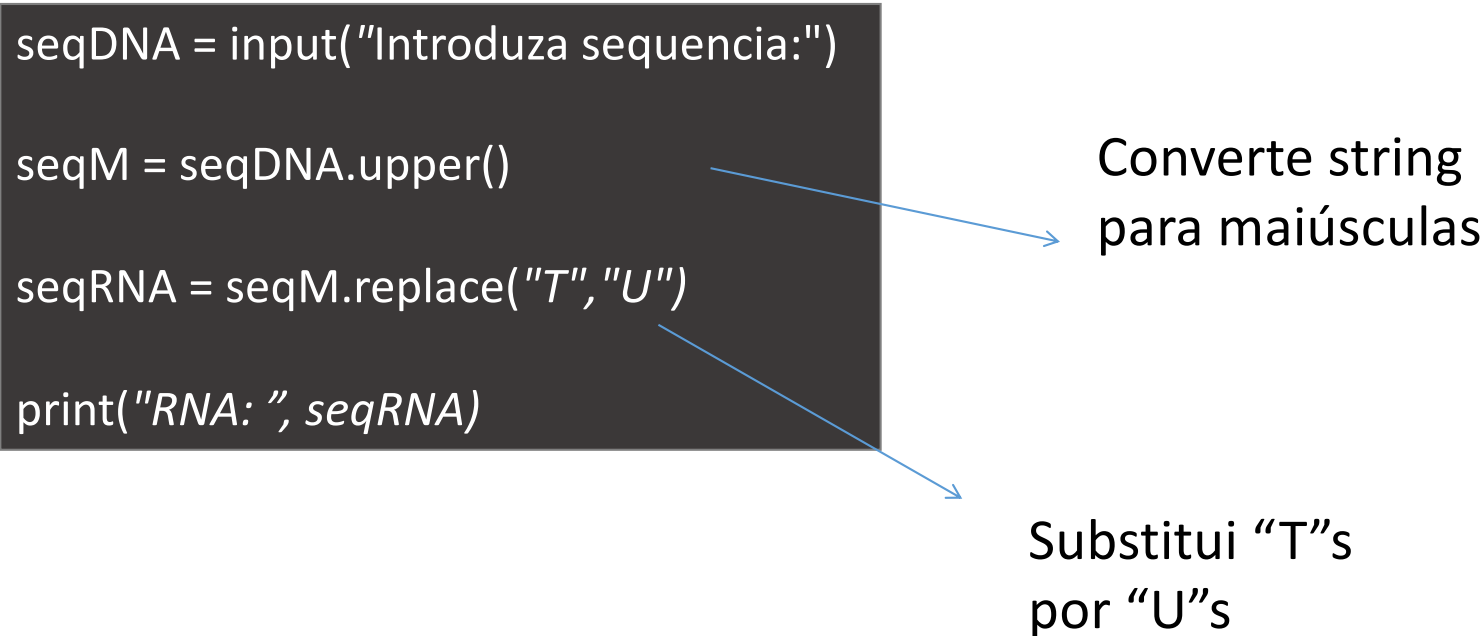
Dado que as sequências de DNA guardadas nas BDs são de coding DNA (cDNA), lidas na direção 5' → 3', basta alterar todos os T's para U's



Programa: transcrição de DNA

```
seqDNA = input("Introduza sequencia:")  
seqM = seqDNA.upper()  
seqRNA = seqM.replace("T", "U")  
print("RNA: ", seqRNA)
```

Converte string
para maiúsculas



Substitui "T"s
por "U"s

Definição de novas funções em python

A definição de uma nova função em Python usa a seguinte sintaxe:

```
def nome_da_funcao ( arg1, arg2, arg3, ... ):
    instrucao 1
    instrucao 2
    (...)
    return resultado
```

Funções que não retornem resultados devem ter:

return None

(embora isto seja opcional)

Note a indentação: todas as instruções dentro do corpo da função devem estar indentados no interior do bloco def

Exemplos de definição de funções

```
def gc_content(base_seq):  
    seq = base_seq.upper()  
    return float(seq.count('G') + seq.count('C')) / len(seq)
```

```
>>> seq50 = 'AACCTTGG'  
>>> seq75 = 'ATCCCGGG'  
>>> gc_content(seq50)  
0.5  
>>> gc_content(seq75)  
0.75
```

shell

Programa

```
seq50 = 'AACCTTGG'  
seq75 = 'ATCCCGGG'  
print (gc_content(seq50))  
print (gc_content(seq75))
```

Assume definição das funções e
programa no mesmo ficheiro

Função para a transcrição

Entrada: string com sequência de DNA

Resultado: string com sequência de RNA

```
def transcricao (dna):  
    return dna.upper().replace("T","U")
```

```
# programa principal  
seq_dna = input("Introduza sequencia:")  
print ("Transcricao: " + transcricao(seq_dna))
```

Esta função e as seguintes devem ser colocadas num único ficheiro, por exemplo com o nome *sequencias.py*

Tradução

A **tradução** consta da conversão de uma sequência de cDNA/mRNA, numa sequência de aminoácidos

O processo passa por:

- identificar dos codões (sub-sequências sucessivas de 3 bases)

- converter cada codão no aminoácido respetivo, segundo a tabela de conversão usada pela espécie

- juntar (concatenar) os aminoácidos resultantes pela ordem dada, para criar as possíveis proteínas

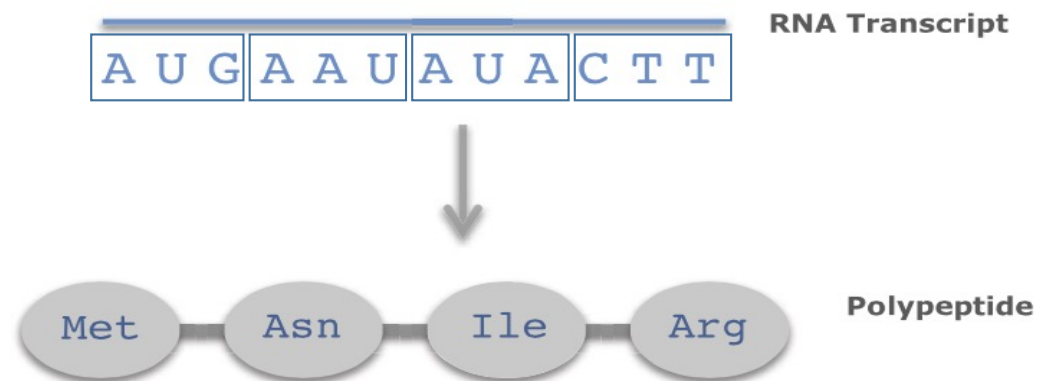


Tabela de conversão: código standard

1st base	2nd base								3rd base
	U		C		A		G		
U	UUU	(Phe/F) Phenylalanine	UCU	(Ser/S) Serine	UAU	(Tyr/Y) Tyrosine	UGU	(Cys/C) Cysteine	U
	UUC		UCC		UAC		UGC		C
	UUA		UCA		UAA	Stop (Ochre)	UGA	Stop (Opal)	A
	UUG		UCG		UAG	Stop (Amber)	UGG	(Trp/W) Tryptophan	G
C	CUU	(Leu/L) Leucine	CCU	(Pro/P) Proline	CAU	(His/H) Histidine	CGU	(Arg/R) Arginine	U
	CUC		CCC		CAC		CGC		C
	CUA		CCA		CAA	(Gln/Q) Glutamine	CGA		A
	CUG		CCG		CAG		CGG		G
A	AUU	(Ile/I) Isoleucine	ACU	(Thr/T) Threonine	AAU	(Asn/N) Asparagine	AGU	(Ser/S) Serine	U
	AUC		ACC		AAC		AGC		C
	AUA		ACA		AAA	(Lys/K) Lysine	AGA	(Arg/R) Arginine	A
	AUG ^[A]	(Met/M) Methionine	ACG		AAG		AGG		G
G	GUU	(Val/V) Valine	GCU	(Ala/A) Alanine	GAU	(Asp/D) Aspartic acid	GGU	(Gly/G) Glycine	U
	GUC		GCC		GAC		GGC		C
	GUA		GCA		GAA	(Glu/E) Glutamic acid	GGA		A
	GUG		GCG		GAG		GGG		G

Programa: tradução de um codão com IF

```
codao = input("Codao:")

if (codao=="GCT" or codao=="GCC" or codao=="GCA"
    or codao=="GCG"):
    aa = "A"
elif (codao=="TGT" or codao=="TGC"):
    aa = "C"
elif (codao=="GAT" or codao=="GAC" or codao=="AAT"
    or codao=="AAC"):
    aa = "D"
(...)
else:
    aa = "erro"

print ("Aminoacido: " + aa)
```

Entrada: codão (string com 3 caracteres)

Função para traduzir um codão: com dicionário

```
def traduzCodao (cod):  
    tc = { "GCT":"A", "GCC":"A", "GCA":"A", "GCC":"A",  
          "TGT":"C", "TGC":"C", "GAT":"D", "GAC":"D",  
          "GAA":"E", "GAG":"E", ...  
    }  
    cod = cod.upper()  
    if cod in tc:  
        aa = tc[cod]  
    else:  
        aa = "X" # erros marcados como X  
    return aa
```

Tradução de um codão com dicionário – dicionário completo

```
tc = { "GCT": "A", "GCC": "A", "GCA": "A", "GCG": "A", "TGT": "C", "TGC": "C",  
      "GAT": "D", "GAC": "D", "GAA": "E", "GAG": "E",  
      "TTT": "F", "TTC": "F", "GGT": "G", "GGC": "G", "GGA": "G", "GGG": "G",  
      "CAT": "H", "CAC": "H", "ATA": "I", "ATT": "I", "ATC": "I",  
      "AAA": "K", "AAG": "K",  
      "TTA": "L", "TTG": "L", "CTT": "L", "CTC": "L", "CTA": "L", "CTG": "L",  
      "ATG": "M", "AAT": "N", "AAC": "N",  
      "CCT": "P", "CCC": "P", "CCA": "P", "CCG": "P", "CAA": "Q", "CAG": "Q",  
      "CGT": "R", "CGC": "R", "CGA": "R", "CGG": "R", "AGA": "R", "AGG": "R",  
      "TCT": "S", "TCC": "S", "TCA": "S", "TCG": "S", "AGT": "S", "AGC": "S",  
      "ACT": "T", "ACC": "T", "ACA": "T", "ACG": "T",  
      "GTT": "V", "GTC": "V", "GTA": "V", "GTG": "V",  
      "TGG": "W", "TAT": "Y", "TAC": "Y",  
      "TAA": "_", "TAG": "_", "TGA": "_  
    }
```

Estruturas cíclicas

Permitem repetir um bloco de instruções várias vezes:

- Um número fixo de vezes
- Um número de vezes determinado por uma condição de terminação

Vários tipos de instruções cíclicas distintas (ciclos **for**, **while**)

Ciclo FOR

```
for var in iterador:  
    instrução 1  
    instrução 2  
    (...)
```



Bloco de instruções
executado para cada valor
da variável

Executa o bloco de instruções para os valores da variável *var* no conjunto dado pelo **iterador** (iterador pode ser uma list, string, set, tuple, ou o resultado de uma função que retorne um iterador como **range**, etc)

Em cada iteração a variável *var* é atualizada para o valor seguinte do iterador

Note-se a indentação do bloco de instruções a realizar dentro do ciclo

Exemplos

```
>>> l = [1,2,3,4,5]
>>> sum(l)
15

>>> soma = 0
>>> for elem in l:
...     soma = soma + elem
...
>>> soma
15
```

Listas por compreensão

```
>>> [2*x for x in range(6)]
[0, 2, 4, 6, 8, 10]
>>> import math
>>> [math.sqrt(x) for x in [1,4,9,16]]
[1.0, 2.0, 3.0, 4.0]
```

A palavra chave **for** pode ser usada no interior de um construtor de listas (ou outros containers) para especificar a lista de elementos de uma lista por compreensão

Exemplo: inverso de uma string com ciclo for

- Entradas: uma sequência (string)
- Saída: a sequência invertida

Versão mais compacta sem ciclos for

```
seq = input("Sequencia:")  
print(seq[::-1])
```

```
seq = input("Sequencia:")  
rev = ""  
for c in seq:  
    rev = c + rev  
print(rev)
```

Inicialmente:
sequência vazia

Acumula símbolos
inserindo na 1ª
posição (à cabeça)

```
(...)  
for i in range(len(seq)):  
    rev += seq[-(i+1)]  
(...)
```

range(n): devolve um iterador com valores de 0 a n-1

```
(...)  
for i in range(len(seq)):  
    rev = seq[i] + rev  
(...)
```

Complemento inverso

Para se obter a sequência **complementar** de uma sequência de DNA deve-se fazer a seguinte conversão, posição a posição:

A => T; T => A; C=> G; G => C

Para manter a leitura na mesma direção a sequência complementar deverá ser invertida, obtendo-se assim o **complemento inverso**.

Permite verificar qual será a outra cadeia do DNA (a que não está guardada nas BDs)



Função para calcular o complemento inverso

Entrada: string com
sequência de DNA

Resultado: string com
sequência complementar
inversa

```
def compinverso(dna):  
    comp = ""  
    for c in dna.upper():  
        if (c == 'A'): comp = "T" + comp  
        elif (c == "T"): comp = "A" + comp  
        elif (c == "G"): comp = "C" + comp  
        elif (c == "C"): comp = "G" + comp  
    return comp
```

Adicionar ao ficheiro *sequencias.py*

Função para traduzir uma sequência de DNA

Entrada: string com a sequência de DNA

Resultado: string com a sequência de aminoácidos

```
def traduzSeq (seqDNA, iniPos= 0):  
    seqM = seqDNA.upper()  
    seqAA = ""  
    for pos in range(iniPos, len(seqM)-2, 3):  
        codao = seqM[pos:pos+3]  
        seqAA += traduzCodao(codao)  
    return seqAA
```

Reading Frames - RFs

Cada sequência de DNA poderá ter **seis frames de leitura** (Reading Frames), na procura de possíveis proteínas codificadas:

Na sequência original podemos começar a ler nas posições 1, 2 ou 3, tendo assim 3 sequências distintas de aminoácidos
Na sequência complementar inversa podemos começar a ler nas posições 1, 2 ou 3.

Assim, de uma sequência de DNA podem extrair-se 6 possíveis sequências de aminoácidos.

Função para realizar a tradução de todas as RFs de uma sequência

Entrada: string com sequência de DNA

Resultado: lista com sequências de aminoácidos de cada RF

```
def orfs (seqDNA):  
    res = [ ]  
    res.append(traduzSeq(seqDNA,0))  
    res.append(traduzSeq(seqDNA,1))  
    res.append(traduzSeq(seqDNA,2))  
    compinv = compinverso(seqDNA)  
    res.append(traduzSeq(compinv,0))  
    res.append(traduzSeq(compinv,1))  
    res.append(traduzSeq(compinv,2))  
    return res
```

Ciclo **WHILE**

while *condição:*
instrução 1
instrução 2
(...)



Bloco de instruções
executado se a condição for
verdadeira

Executa um bloco de instruções enquanto a
condição se mantém verdadeira (0 ou + vezes)

Condições semelhantes às do if (valor lógico)

Note a indentação do bloco de instruções dentro
do while

```
>>> a=10
>>> while a<40:
...     print(a)
...     a = a+10
...
10
20
30
```

Construindo um ciclo FOR com um ciclo WHILE

```
for ind in range(n):  
    <bloco de instruções>
```



```
ind = 0  
while (ind < n):  
    <bloco de instruções>  
    ind = ind + 1
```

Procurar padrões em sequências: algoritmo

Se quisermos procurar um padrão fixo de comprimento L numa sequência de tamanho N , um algoritmo base será:

Percorrer todas as possíveis posições iniciais (0 a $N-L$)

Para cada uma destas posições verificar se a sub-sequência de tamanho L iniciada nessa posição corresponde ao padrão

Função para procurar um padrão numa sequência

Entradas: string com sequência; padrão a procurar

Resultado: posição onde ocorre padrão (-1 se não ocorre)

```
def procuraPadrao (seq, pad):  
    res = -1  
    pos = 0  
    while pos <= (len(seq) - len(pad)) and res < 0:  
        if seq[pos:(pos+len(pad))] == pad:  
            res = pos  
        else: pos += 1  
    return res
```

Função para procurar um padrão numa sequência: todas as ocorrências

Entradas: string com sequência; padrão a procurar

Resultado: posição onde ocorre padrão (-1 se não ocorre)

```
def procuraPadraoTodas (seq, pad):  
    res = [ ]  
    for pos in range(len(seq) - len(pad) + 1):  
        if seq[pos:(pos+len(pad))] == pad:  
            res.append(pos)  
    return res
```

Possíveis proteínas

Todas as possíveis proteínas iniciam-se pelo aminoácido Metionina (M) e terminam com stop (representado por _) – **M** _

O stop não pode ocorrer no interior de uma proteína; a Metionina por sua vez pode ocorrer dentro de uma proteína

Seguindo a regra anterior, é possível extrair de uma sequência de aminoácidos todas as possíveis proteínas no seu interior (a haver alguma)

Podemos também aplicar o mesmo processo a todas as reading frames geradas por uma sequência de DNA, obtendo assim as Open Reading Frames

Obviamente, quanto maior for a potencial proteína, maior será a probabilidade de ser realmente uma proteína

Função para identificar proteínas

Entrada: uma sequência de aminoácidos

Saída: lista de todas as possíveis proteínas contidas
nessa sequência

Escrita esta, poderá criar uma função para identificar proteínas em
todas as ORFs

Função para identificar proteínas – 1ª tentativa

```
def todasProteinas (seqAA):  
    seqAA = seqAA.upper()  
    dentroProt = False  
    protAtual = ""  
    proteinas = [ ]  
    for aa in seqAA:  
        if dentroProt:  
            if aa == "_":  
                dentroProt = False  
                proteinas.append(protAtual)  
            else: protAtual += aa  
        else:  
            if aa == "M":  
                dentroProt=True  
                protAtual = "M"  
    return proteinas
```

Percorre uma única vez a sequência com ciclo for

Usa variável *dentroProt* para controlar estado

Porque não resolve o problema ?

Função para identificar proteínas – 2ª tentativa

```
def todasProteinas_v2 (seqAA):  
    seqAA = seqAA.upper()  
    protsAtuais = []  
    proteinas = []  
    for aa in seqAA:  
        if aa == "_":  
            if protsAtuais:  
                for p in protsAtuais:  
                    proteinas.append(p)  
                protsAtuais = []  
            else:  
                if aa == "M":  
                    protsAtuais.append("")  
                for i in range(len(protsAtuais)):  
                    protsAtuais[i] += aa  
    return proteinas
```

***Qual a principal
diferença ?***

Função para identificar todas as proteínas em todas as RFs

```
def todasProteinasORFs (seqDNA):  
    porfs = orfs(seqDNA)  
    res = [ ]  
    for orf in porfs:  
        prots = todasProteinas(orf)  
        for p in prots:  
            res.append(p)  
    return res
```